

1a) We can generate coredump files automatically by setting a ulimit (preferably unlimited) and making sure the process has permission to create the coredump file. Then, kill with signal 3 (quit) generates a coredump file on termination. There is also a gdb utility* (gcore) which, when the program PID is ran with, creates a coredump in the current directory.

1b) We can use **stty*** to remap backsapce to the interrupt signal. The command ``stty intr <x>`` remaps <x> to send the SIGINT signal, which in our case we would want <x> to be ascii 8 or \b (backspace).

1c) A specific situation is one in which you write to a pipe but the pipe (read pipe open in a child process for example) breaks due to the child process terminating from a signal handler. This is a SIGCHLD since it represents a child process changing state. Then, we have only partially written but it is assumed that we've written our whole data. So technically, our output is corrupt.

1d) The process is in an uninterruptable state, and it may be performing disk I/O operations that are hanging due to a bad disk sector.

1e) On multi-cpu systems since processes may be "Running" on different cores than the signal sending core, the system can take advantage of inter-processor interrupts (also apparently known as shoulder taps) from the signal sending core which reminds/forces the target core to run an interrupt handler, thereby entering kernel mode. In order for a core to send a signal to a process running on another core, an IPI is used to deliver this signal (but it does not represent the signal itself).

2a)

At 1000 -> reports 187

At 2000 -> reports 462

At 5000 -> reports 1277

The reported count is always about 20-25% of the input count. This is because after a signal is delivered, pending signals of the same type are cleared. Signal 15 is SIGTERM,

1. Sourced from
Kenneth Chan
who wonderfully
asked Professor
Hakner on Zidane
Karim's behalf

2. <https://stackoverflow.com/questions/318647/what-is-a-good-way-to-dump-a-linux-core-file-from-inside-a-process>

which is NOT a real time signal, so these signals are not queued, and many of these signals are ignored/dropped, so we receive less signals than we expected.

2b) The parent prints a wait status of 8b which means the child is segfaulting. This is because the flag SA_NOMASK tells the sigaction to not block the signal/add it to the bitmask while it is being handled. This in turn allows the stack frame to pile up as multiple interrupt signals are sent, and eventually this stack frame corrupts from being overloaded, resulting in the seg fault.

2c) It no longer seg faults because the signal is properly blocked during handling so the program waits instead of spamming the stack frame.

2d) The count gets closer and closer to the input count as the signal number goes up. (On my machine, this only reached the input count when I went past 50). This is because these are the real-time/queued signals, so there is actually a line/waiting for these signals to arrive rather than discarding them.